

Guión de bolsillo — Defensa MaduraApp

~14 min · Habla con TUS palabras, no leas todo. Mira al jurado en las (CLAVE).

Antes de empezar (en la sala)

- **Start Lab** → EC2 → **Iniciar instancia** → esperar 2-3 min
 - Probar en el navegador del teléfono:
`http://3.215.43.61:8000/v1/health` → **model_loaded:true**
 - App lista en el teléfono
-

1 · Portada (~20s)

- Saludo + tu nombre
- “Defensa del Estado de Avance 3 de MaduraApp”
- “Hoy verán: el plan de pruebas, cómo lo apliqué y las mejoras”

2 · Agenda (~20s)

- Contexto → plan de pruebas → aplicación y resultados → mejoras → conclusión

3 · Contexto (~1.5 min)

- Problema: **20-40%** de pérdida post-cosecha (FAO/ODEPA)
- Causa raíz: no hay criterio **objetivo** de madurez
- Solución: una foto → estado (inmaduro / óptimo / sobremaduro) + recomendación
- 4 frutas: aguacate, plátano, tomate, mango
- Stack: Android (Kotlin) + API FastAPI + modelo **YOLO26n**
- Precisión **mAP@50 = 0.92** (meta era 0.75)

4 · Plan de pruebas (~2.5 min) (CLAVE)

- **36 casos** automatizados → 17 backend + 19 Android
- 4 tipos:
 - – **Validación** = ¿el sistema correcto? (requisitos)
 - – **Verificación** = ¿correctamente? (calidad)
 - – **Seguridad** = OWASP
 - – **Operacional** = funciona en su entorno
- BD de pruebas: **SQLite en memoria** → aísla, rápido, sin datos reales

5 · Aplicación y resultados (~2 min)

- **36/36 en verde** (ejecutadas hoy)
- backend 17/17 (pytest) · Android 19/19 (MockK + JUnit)
- **Integración continua:** la suite corre sola en cada cambio
- Calidad: modelo 0.92 · 5.2 MB · APK compila

6 · Mejoras (~2.5 min) (CLAVE)

- “Las mejoras **no son arbitrarias**: salen de las pruebas + auditoría OWASP”
- **Seguridad:** secreto JWT fuera del código · HTTPS · token cifrado **AES-256**
- **Usabilidad:** rediseño Material 3 · modo oscuro · íconos vectoriales
- **Corrección:** suite y compilación en verde
- **Compleitud:** autenticación + feedback end-to-end
- **Pertinencia:** recomendaciones del dominio agrícola
- “Cada mejora tiene un **commit** → trazable”

7 · Seguridad (~1.5 min)

- bcrypt (hash de contraseñas) · JWT · HTTPS (tránsito) · AES-256 (reposo)
- (CLAVE) **Ética:** “No le pongo 'extremo a extremo' como WhatsApp porque **sería falso** — mi servidor necesita leer la imagen. Uso 'cifrado en tránsito'. La honestidad también es calidad.”

8 · Conclusión (~1.5 min) (CLAVE)

- Producto **funcional, probado y seguro**
- **36/36** pruebas · **15 mejoras** · datos protegidos
- Probado en producción: backend en **AWS (EC2 + Docker)**, escaneé un **plátano real** → **Óptimo**
- Pendiente honesto: pruebas E2E + su **aprobación del plan** hoy

9 · Cierre (~10s)

- “Eso es todo. Gracias por su atención. Quedo atento a sus preguntas.”

Si me preguntan (respuestas rápidas)

- ¿**SQLite en memoria**? Aísla cada test, rápido, sin datos reales.

- **¿Por qué mockean el modelo?** El test valida la lógica de la API, no la red neuronal. La precisión se mide aparte (mAP).
- **¿Validación vs verificación?** Validación = sistema correcto. Verificación = correctamente (calidad).
- **¿Por qué no E2E como WhatsApp?** El servidor necesita leer la imagen; sería falso. Es cifrado en tránsito + reposo.
- **¿Qué es OWASP?** El estándar de los 10 riesgos de seguridad más críticos en apps.
- **¿Qué es mAP@50?** Métrica de precisión del modelo; 0.92 es muy alta (meta 0.75).
- **¿Qué falta?** Automatizar pruebas E2E y un despliegue permanente; el lab AWS es temporal.

Tips: respira en los puntos · mira al jurado en las (CLAVE) · si no sabes algo, reconócelo y explica cómo lo averiguarías.